

Министерство образования Республики Беларусь
Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет информационных технологий и управления

Кафедра информационных технологий автоматизированных систем

Дисциплина интернет-технологии и веб-программирование

ОТЧЁТ
к лабораторной работе
на тему

**НАСТРОЙКА РАБОЧЕГО ОКРУЖЕНИЯ ДЛЯ ВЕБ-
РАЗРАБОТКИ**

Вариант 3

Выполнили

В. М. Равгейша
А. А. Почиковский

Проверил

Н. В. Хаджинова

Минск 2026

СОДЕРЖАНИЕ

1	Цель работы	3
2	Краткие теоритические сведения	4
3	Ход работы	7
4	Ответы на контрольные вопросы	11
4.1	Основные преимущества использования <i>VS Code</i>	11
4.2	Интеграция <i>Git</i> в процесс разработки в <i>VS Code</i>	11
4.3	Роль <i>npm</i> и <i>Node.js</i> в веб-разработке	11
4.4	Интеграция <i>Figma</i> в процесс разработк.....	12
4.5	Настройки <i>VS Code</i> . Организация рабочего пространства	12
	Вывод.....	13

1 ЦЕЛЬ РАБОТЫ

Целью данной лабораторной работы является формирование и закрепление комплексных практических навыков по развертыванию и конфигурированию современной программной инфраструктуры, необходимой для полноценного цикла веб-разработки. В рамках работы предполагается глубокое изучение и освоение распределенных систем контроля версий на базе *Git*, также механизмов взаимодействия с удаленными облачными репозиториями на платформе *GitHub*.

Важной задачей является освоение процессов инициализации проекта, построение его логической структуры и управление внешними зависимостями с использованием среды выполнения *Node.js* и пакетного менеджера *npm*. Работа направлена на детальное изучение функциональных возможностей интегрированной среды разработки *VS Code*, включая настройку специализированных расширений для автоматизации написания, линтинга и форматирования кода, таких как *Prettier*, *ESLint* и *Live Server*.

Дополнительно целью работы является изучение инструментов графического проектирования в среде *Figma*, приобретение навыков подготовки макетов к верстке и использование инструментов интеграции дизайна с программным кодом. В конечном итоге работа призвана сформировать понимание единого технологического процесса создания веб-приложений: от настройки локального окружения и проектирования интерфейса до фиксации изменений и публикации проекта в сети.

2 КРАТКИЕ ТЕОРИТИЧЕСКИЕ СВЕДЕНИЯ

Процесс создания современных программных продуктов невозможно представить без использования систем контроля версий, среди которых лидирующее положение занимает *Git*. Это распределенная система, которая позволяет разработчикам фиксировать каждое изменение в коде, обеспечивая возможность возврата к любому предыдущему состоянию проекта. В отличие от старых централизованных систем, в *Git* каждый участник команды обладает полной локальной копией всего репозитория. Это делает работу чрезвычайно гибкой, позволяя выполнять большинство операций, таких как *commit* или создание веток, локально и с очень высокой скоростью. Основная логика работы строится на взаимодействии трех зон: рабочего каталога, области подготовки под названием *staging area* и непосредственно самого репозитория, где хранятся снимки состояния файлов.

Для эффективного управления проектом разработчики используют набор базовых команд. Процесс обычно начинается с инициализации проекта через *git init* или клонирования существующего кода с помощью *git clone*. В процессе работы изменения подготавливаются командой *git add* и фиксируются через *git commit* с обязательным поясняющим сообщением. Для обмена данными с удаленным сервером применяются команды *git push* и *git pull*. Важнейшей особенностью является механизм ветвления, где основная стабильная версия кода хранится в ветке *main* или *master*, а новые функции разрабатываются в отдельных ветках, которые впоследствии объединяются через *git merge*. Такой подход минимизирует риски поломки основного продукта и позволяет множеству специалистов работать над разными частями системы одновременно.

Для размещения репозитория и организации совместной работы используются такие платформы, как *GitHub* и *GitLab*. *GitHub* является крупнейшим в мире облачным сервисом для хостинга кода, который принес в индустрию культуру социального программирования. Его ключевым инструментом стал *pull request* – запрос на слияние изменений, в рамках которого коллеги проводят проверку кода, обсуждают правки и только после одобрения интегрируют их в основную ветку. Кроме того, сервис предлагает такие возможности, как *GitHub Actions* для автоматизации процессов, *GitHub Pages* для размещения статических сайтов и встроенные системы отслеживания задач, называемые *issues*.

Жизненный цикл веб-приложения начинается задолго до написания первой строки кода. На этапе планирования бизнес-аналитики и менеджеры формируют *user stories*, описывающие потребности пользователей. Для управления этим процессом применяются инструменты вроде *Jira*, *Trello* или *Notion*. Затем следует этап проектирования архитектуры и дизайна, где создаются макеты в *Figma*. Современные дизайнеры создают целые дизайн-системы, которые затем переводятся в код с использованием принципов адаптивности и доступности, таких как стандарты *WCAG*.

Технологический стек разработки традиционно делится на *frontend* и *backend*. Фронтенд-разработчик отвечает за все, что видит пользователь, используя связку из *HTML5*, *CSS3* и *JavaScript*. В современной практике вместо чистого языка часто используется *TypeScript*, который добавляет статическую типизацию и помогает избегать ошибок. Основными инструментами здесь выступают библиотеки и фреймворки: *React.js*, *Vue.js*, *Angular* или *Svelte*. Для оформления интерфейсов активно применяются такие методологии, как *BEM*, и современные инструменты вроде *Tailwind CSS*. Сборка приложения осуществляется с помощью быстрых инструментов, таких как *Vite* или *Webpack*.

Серверная часть, или *backend*, отвечает за бизнес-логику и работу с базами данных. Выбор языка здесь зависит от задач: *Node.js* на базе *JavaScript* идеален для приложений реального времени, *Python* с фреймворками *Django* или *FastAPI* лидирует в сфере аналитики и быстрой разработки, а языки *Go* и *Rust* обеспечивают высочайшую производительность в микросервисной архитектуре. Данные обычно хранятся в реляционных базах, таких как *PostgreSQL* или *MySQL*, либо в нереляционных хранилищах типа *MongoDB* и *Redis*. Взаимодействие между частями системы происходит через *API*, построенные по стандартам *REST* или *GraphQL*.

Важным этапом является тестирование и обеспечение качества, известное как *QA*. Разработчики пишут юнит-тесты с помощью *Jest* или *Vitest*, а также проводят комплексные проверки сценариев через *Cypress* или *Playwright*. После успешных тестов наступает этап деплоя, когда приложение разворачивается на облачных платформах вроде *AWS*, *Google Cloud* или *Vercel*. Процесс автоматизируется с помощью контейнеризации в *Docker* и систем оркестрации вроде *Kubernetes*. Для мониторинга стабильности системы в реальном времени используются инструменты логирования и анализа ошибок, такие как *Sentry* или *Prometheus*.

Инструментарий разработчика постоянно развивается. Основной рабочей средой сегодня является *IDE*, чаще всего *VS Code*, дополненный множеством расширений для линтинга кода, таких как *ESLint*, и автоматического форматирования через *Prettier*. В 2024 году важным трендом стало использование искусственного интеллекта, например *GitHub Copilot*, для генерации шаблонного кода. Также набирают популярность архитектуры *JAMstack* для статических сайтов и подход *serverless*, позволяющий запускать функции без управления серверами. Весь этот набор технологий и методологий направлен на ускорение разработки и повышение надежности конечного веб-продукта.

Процесс подготовки приложения к запуску обеспечивают сборщики, среди которых лидируют *Vite* и *Webpack*. Эти инструменты оптимизируют файлы для браузера и обеспечивают быструю работу интерфейса. Управление всеми внешними библиотеками, такими как *Axios* для запросов или препроцессоры *Sass*, осуществляется через менеджеры пакетов *npm* или *Yarn*, что позволяет легко подключать к проекту готовые решения.

Командное взаимодействие строится вокруг системы контроля версий *Git*, которая позволяет отслеживать каждое изменение через механизм веток и коммитов. Хранение кода и совместный аудит правок происходят на облачных платформах *GitHub*, *GitLab* или *Bitbucket*. Здесь же настраиваются процессы *CI/CD* с помощью *Jenkins* или *Travis CI*, которые берут на себя автоматическую сборку и тестирование кода перед его выпуском.

Визуальная концепция приложения рождается в инструментах дизайна, таких как *Figma* или *Adobe XD*, где проектируются интерфейсы. Для последующей отладки и поиска ошибок в реальном времени незаменимы браузерные инструменты *Chrome DevTools*, а за проверку надежности функционала отвечают фреймворки для тестирования *Jest* или *Cypress*. Это гарантирует, что программная логика работает корректно в любых условиях.

Финальным звеном в структуре разработки выступают системы управления проектами, такие как *Jira* или *Trello*. Они позволяют командам работать по гибким методологиям *Agile*, планировать задачи в спринтах и прозрачно отслеживать прогресс каждого участника. Весь этот комплексный инструментарий в совокупности направлен на то, чтобы сделать процесс создания веб-приложений предсказуемым, быстрым и защищенным от ошибок.

3 ХОД РАБОТЫ

Для начала работы была произведена установка актуальной LTS-версии *Node.js* с официального сайта. После завершения установки необходимо открыть терминал внутри *Visual Studio Code*, чтобы убедиться в корректности установки. С помощью команд `node --version` и `npm --version` были получены данные о версиях среды выполнения и пакетного менеджера.

```
• → itivp git:(lab2-implementation) x npm -v
11.6.2
• → itivp git:(lab2-implementation) x node -v
v24.13.0
○ → itivp git:(lab2-implementation) x
```

Рисунок 1 – Проверка версий *Node.js* и *npm*

Затем был установлен *Git*. После установки в терминале были настроены глобальные параметры пользователя (имя и *email*), чтобы все будущие коммиты корректно идентифицировались. Также была выполнена установка самого редактора *Visual Studio Code* и создание рабочей учетной записи в сервисе *Figma* для дальнейшего проектирования интерфейса.

Далее необходимо к подготовке редактора. Был изучен интерфейс: панель активности, проводник и встроенный терминал. Для повышения эффективности разработки были установлены расширения: *Live Server* для автообновления страницы в браузере, *Prettier* для форматирования кода, а также *GitLens*. На рисунке 2 показан интерфейс *VS Code* с открытым проектом и установленными расширениями.

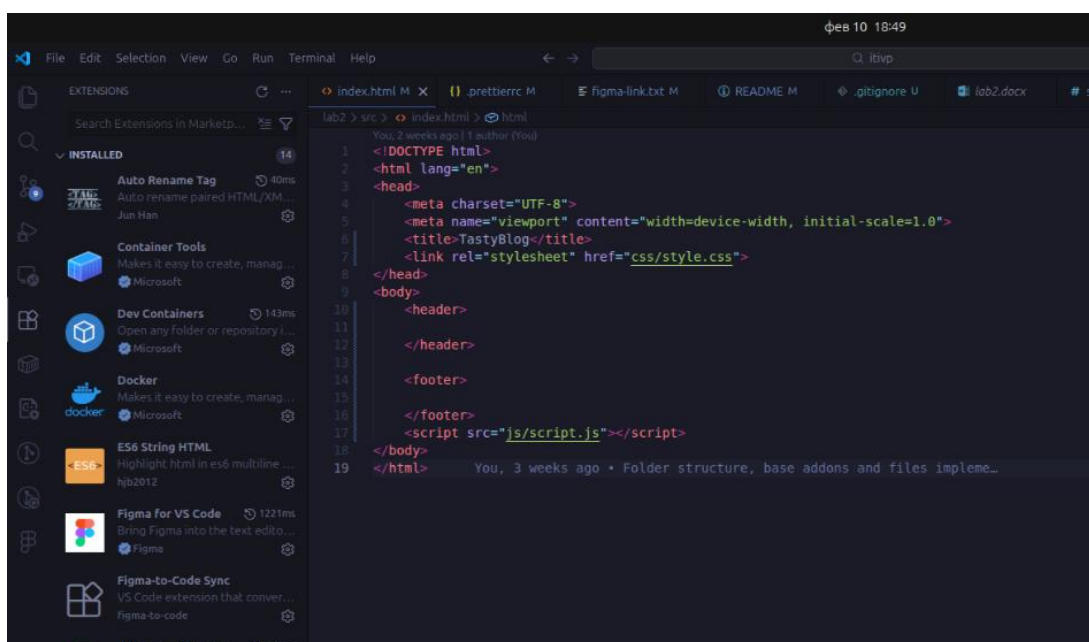


Рисунок 2 – Интерфейс *VS Code*

Следующим заданием было создание проекта. Была создана рабочая директория, внутри которой через терминал командой *mkdir* была развернута стандартную структуру папок: *css*, *js*, *assets*, *images* и *design*. В корне проекта были созданы базовые файлы: *index.html* с первичной разметкой, *style.css* и *script.js*. На рисунке 3 показана структура нового проекта.

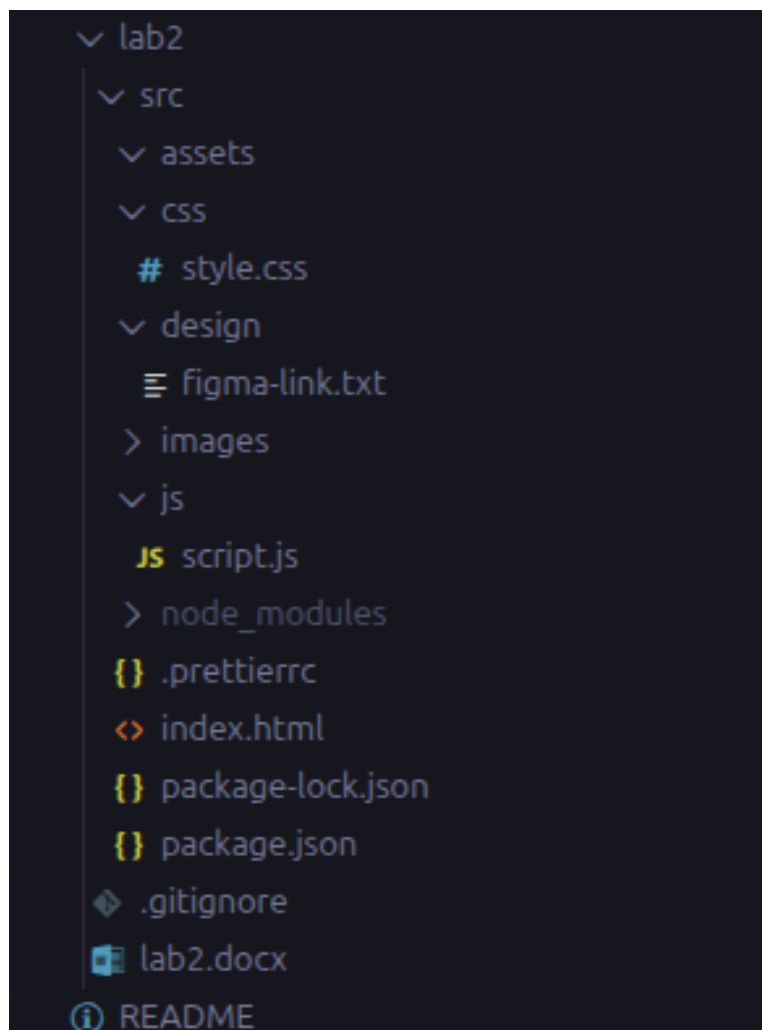


Рисунок 3 – Структура проекта

Параллельно велась работа в *Figma*. Там был создан новый файл, в котором был разработан мудборд, определили цветовую палитру и подготовили прототип главной страницы будущего приложения. На рисунке 4 представлен прототип главной страницы.

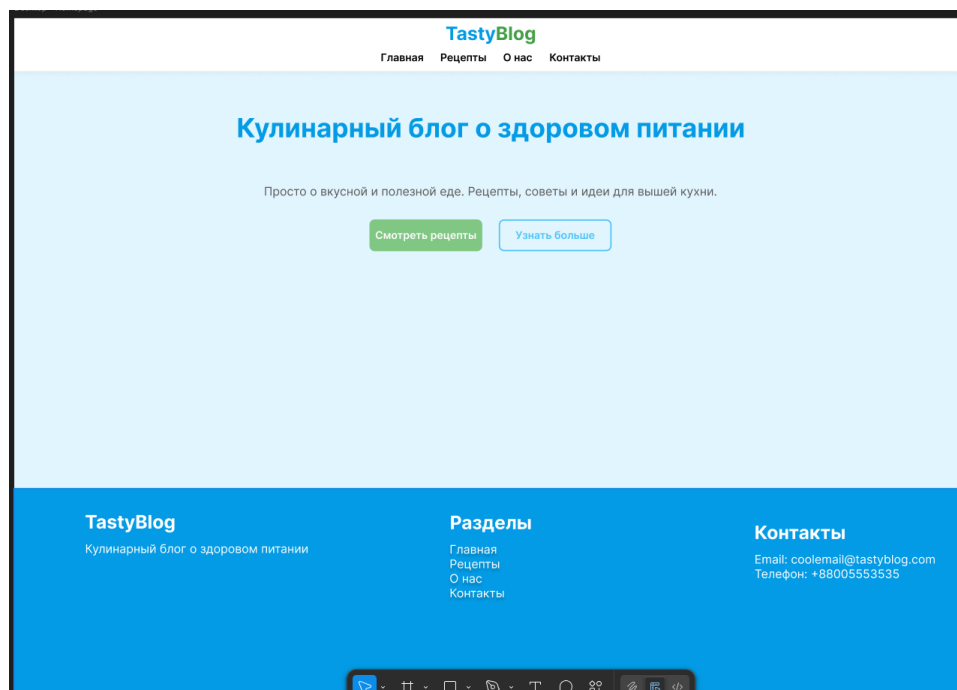


Рисунок 4 – *Figma*-макет с прототипом страницы

Для управления изменениями сделана инициализация локальный репозиторий командой `git init`. Чтобы исключить лишние файлы из отслеживания, был создан файл `.gitignore`, была прописана папку `node_modules` и системные логи. Первый коммит с сообщением «*Initial commit*» был выполнен через графический интерфейс вкладки *Source Control*. На рисунке 5 представлена вкладка *Source Control* с историей коммитов.

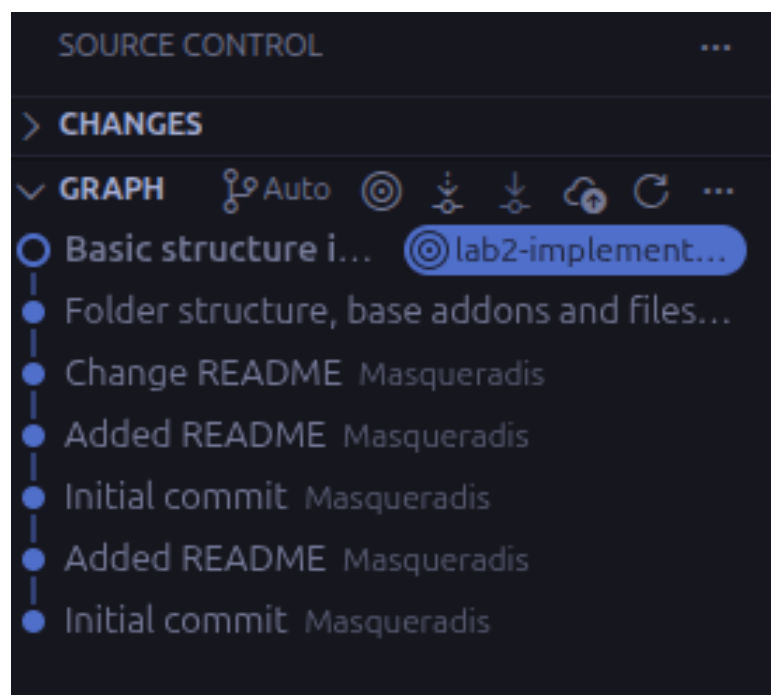


Рисунок 5 – Вкладка *Source Control* с историей коммитов

После этого был создан удаленный репозиторий на платформе *GitHub*, привязали его к локальному проекту командой *git remote add origin* и отправили все файлы в облако с помощью команды *git push*. На рисунке 6 виден репозиторий, загруженный на *GitHub*.

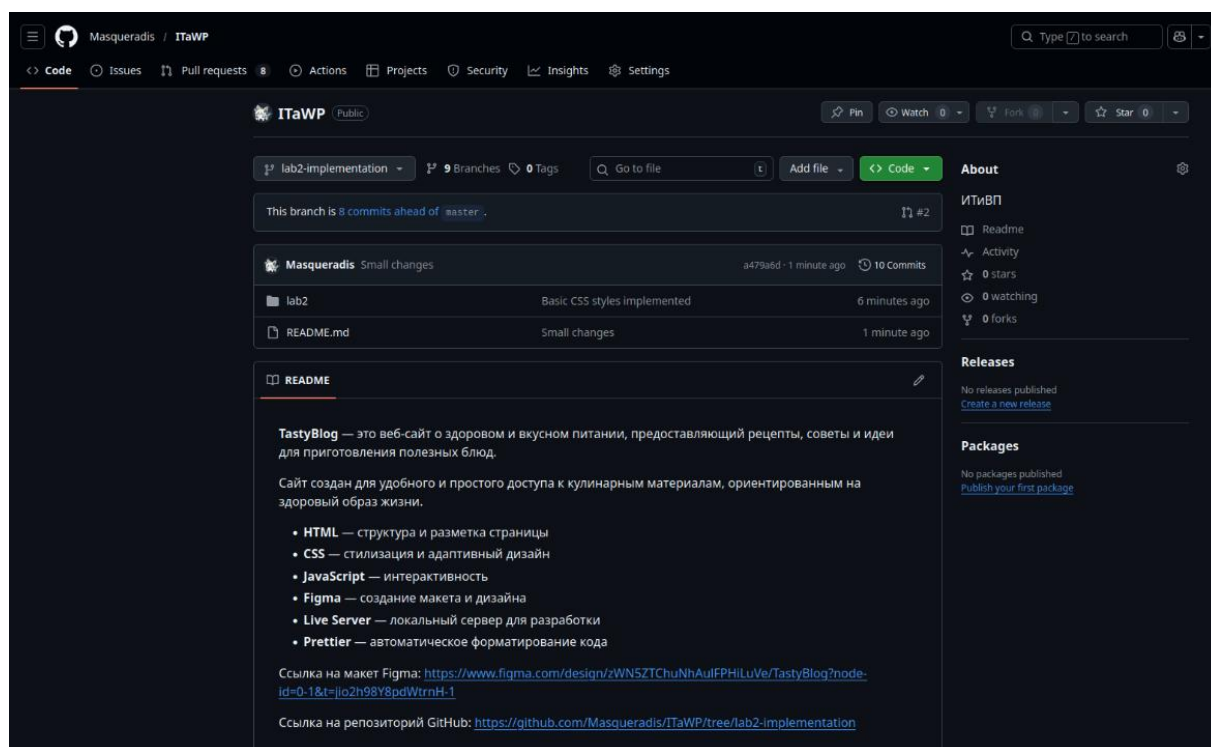


Рисунок 6 – Репозиторий на *GitHub*

Для управления зависимостями проекта была выполнена инициализация *npm*-проекта командой *npm init -y*. После была установлена библиотека *lodash* и инструмент *nodemon* для упрощения разработки. Также была настроена автоматизация форматирования: создан файл конфигурации *.prettierrc*, а в настройках самого редактора активирована функция форматирования при сохранении (*Format on Save*). Ссылка на макет из *Figma* была бережно сохранена в текстовом файле в папке *design*.

4 ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ

4.1 Основные преимущества использования *VS Code*

Основное преимущество *VS Code* заключается в том, что это не просто текстовый редактор, а полноценная экосистема, объединяющая в одном окне написание кода, управление файлами, работу с терминалом и отладку. В отличие от простых редакторов, *VS Code* обладает мощной системой интеллектуальных подсказок *IntelliSense*, которая предугадывает ввод тегов и функций, значительно сокращая количество синтаксических ошибок.

Наличие встроенного терминала позволяет выполнять команды *npm* или *git*, не переключаясь между окнами, что критически важно для поддержания темпа разработки. При настройке текущего проекта наиболее полезными оказались расширения *Live Server*, которое обеспечивает мгновенное обновление страницы в браузере при сохранении изменений, и *Prettier*, гарантирующее аккуратный и единообразный стиль кода во всех файлах. Также важную роль сыграло расширение *GitLens*, предоставляющее наглядную информацию об истории каждой строки кода непосредственно в редакторе.

4.2 Интеграция *Git* в процесс разработки в *VS Code*

Интеграция *Git* в *VS Code* реализована через специальную панель *Source Control*, которая визуально отображает все изменения в файлах проекта. Работа начинается с инициализации репозитория командой *git init* в терминале, после чего редактор начинает отслеживать состояние каждого файла. Основные этапы включают подготовку изменений, когда разработчик помечает файлы знаком «плюс» в интерфейсе, что соответствует команде *git add*, и последующую фиксацию через *commit* с обязательным текстовым пояснением. Использование графического интерфейса в этом случае гораздо удобнее для просмотра разницы в коде между версиями. Синхронизация же с удаленным сервером на *GitHub* чаще всего выполняется через терминал командами *git push* и *git pull*, что обеспечивает обмен кодом с внешним.

4.3 Роль *npm* и *Node.js* в веб-разработке

Node.js выступает в роли фундаментальной платформы, которая позволяет запускать инструменты разработки и автоматизации, написанные на *JavaScript*, вне браузера. Пакетный менеджер *npm*, входящий в её состав, является глобальным хранилищем готовых модулей и библиотек, которые разработчик может легко подключить к своему проекту одной командой. Это избавляет от необходимости вручную искать, скачивать и обновлять файлы библиотек. Главное преимущество менеджера пакетов заключается в автоматическом управлении зависимостями: он контролирует версии установленных инструментов и гарантирует, что проект будет работать

одинаково на любом компьютере. Файл *package.json* служит своего рода паспортом проекта, позволяя мгновенно развернуть рабочее окружение со всеми необходимыми пакетами простой командой *npm install*.

4.4 Интеграция *Figma* в процесс разработки

Figma является связующим звеном между этапом визуального проектирования и непосредственным написанием кода, позволяя разработчику получить точные параметры интерфейса. Вместо того чтобы определять цвета и размеры элементов «на глаз», в режиме инспектора можно увидеть точные значения отступов, размеры шрифтов в пикселях и даже скопировать готовые *CSS*-свойства. Это значительно ускоряет процесс верстки и сводит к минимуму расхождения с макетом. На этапе подготовки к верстке наиболее полезными возможностями *Figma* являются инструменты для экспорта графических ресурсов, таких как иконки в формате *SVG* или изображения в *PNG*, а также возможность изучения интерактивных прототипов, которые наглядно демонстрируют логику переходов и поведение элементов интерфейса при взаимодействии с пользователем.

Дополнительную эффективность обеспечивает использование специализированных расширений, таких как *Figma to Code*. Этот инструмент позволяет автоматизировать процесс рутинного написания кода, преобразуя выбранные элементы дизайна в готовую разметку *HTML* или стили *Tailwind CSS*. Такая интеграция позволяет разработчику не тратить время на базовую верстку структуры, а сразу переходить к реализации сложной логики и интеграции данных, сохраняя при этом идеальное соответствие исходному дизайн-макету.

4.5 Настройки *VS Code*. Организация рабочего пространства

Для достижения максимальной эффективности в *VS Code* первостепенное значение имеет настройка функции *Format on Save*, которая в связке с *Prettier* автоматически «причесывает» код при каждом сохранении файла. Также крайне важно правильно настроить интегрированный терминал, выбрав подходящую оболочку, например *Git Bash*, чтобы команды управления проектом выполнялись корректно. Комфортная организация рабочего пространства предполагает использование режима разделенного экрана, где в одной части окна открыт *HTML*-файл, а в другой – стили *CSS*, что позволяет сразу видеть взаимосвязи между структурой и оформлением. Грамотная структура папок проекта, разделенная на исходные файлы кода, изображения и активы, вместе с использованием миникарты кода и навигационных цепочек, позволяет разработчику быстро ориентироваться в проекте и не терять продуктивность при росте объема кода.

ВЫВОДЫ

В ходе выполнения данной лабораторной работы были успешно реализованы все поставленные задачи по формированию и настройке профессионального рабочего окружения для веб-разработки.

Особое внимание в работе было уделено системе контроля версий *Git*. Была сделана инициализация локального репозитория через *git init* и последующая синхронизация с облачным хостингом *GitHub*, что позволяет не только сохранять историю изменений, но и обеспечивают надежность разработки за счет возможности отката к предыдущим состояниям кода. Создание файла *.gitignore* позволило оптимизировать структуру репозитория, исключив из отслеживания служебные файлы и тяжелые зависимости (*node_modules*), что является критически важным аспектом при работе в команде.

Изучение возможностей интегрированной среды разработки *VS Code* показало, что современная *IDE* значительно превосходит простые текстовые редакторы за счет автоматизации рутинных процессов. Установка и настройка расширений, таких как *Prettier* и *Live Server*, позволили реализовать механизм «чистого кода» и мгновенного отображения результатов верстки в браузере. Настройка функции *Format on Save* гарантирует соблюдение стандартов оформления кода без дополнительных усилий со стороны разработчика, что повышает читаемость и поддерживаемость программного продукта.

Работа с пакетным менеджером *npm* и средой *Node.js* продемонстрировала механизмы управления зависимостями современного веб-приложения. Инициализация проекта и установка сторонних библиотек позволили понять, как строятся модульные системы и как эффективно использовать открытые решения для ускорения процесса разработки.

Важным этапом работы стала интеграция инструментов проектирования в цикл разработки. Использование сервиса *Figma* для создания мудбордов и интерактивных прототипов позволило структурировать визуальную концепцию проекта еще до начала написания кода. Мы освоили методы извлечения точных параметров интерфейса (цветов, шрифтов, отступов) и экспорт графических ресурсов, что минимизирует расхождения между дизайном и конечной версткой. Применение расширения *Figma to Code* подтвердило тенденцию к сближению процессов проектирования и программирования.